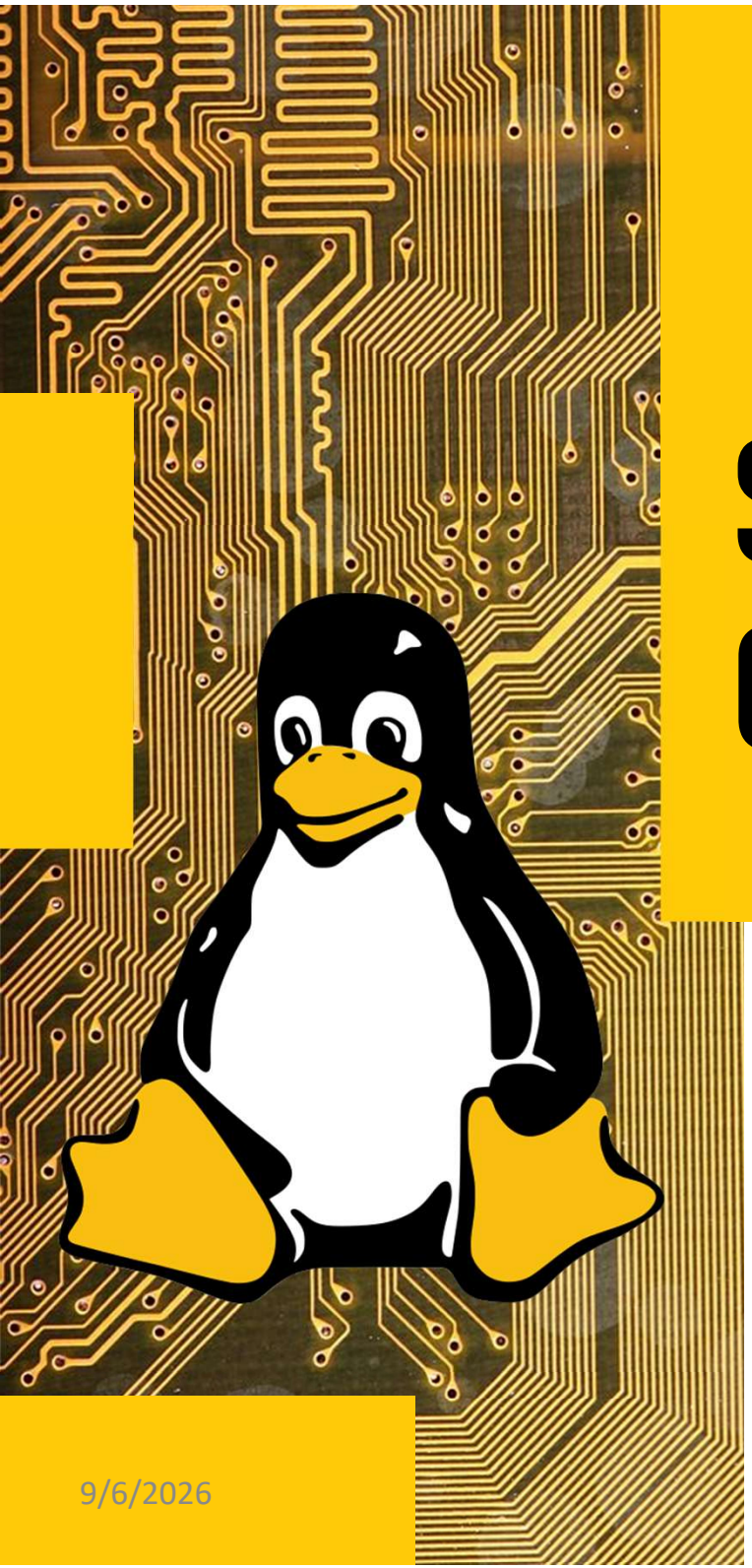


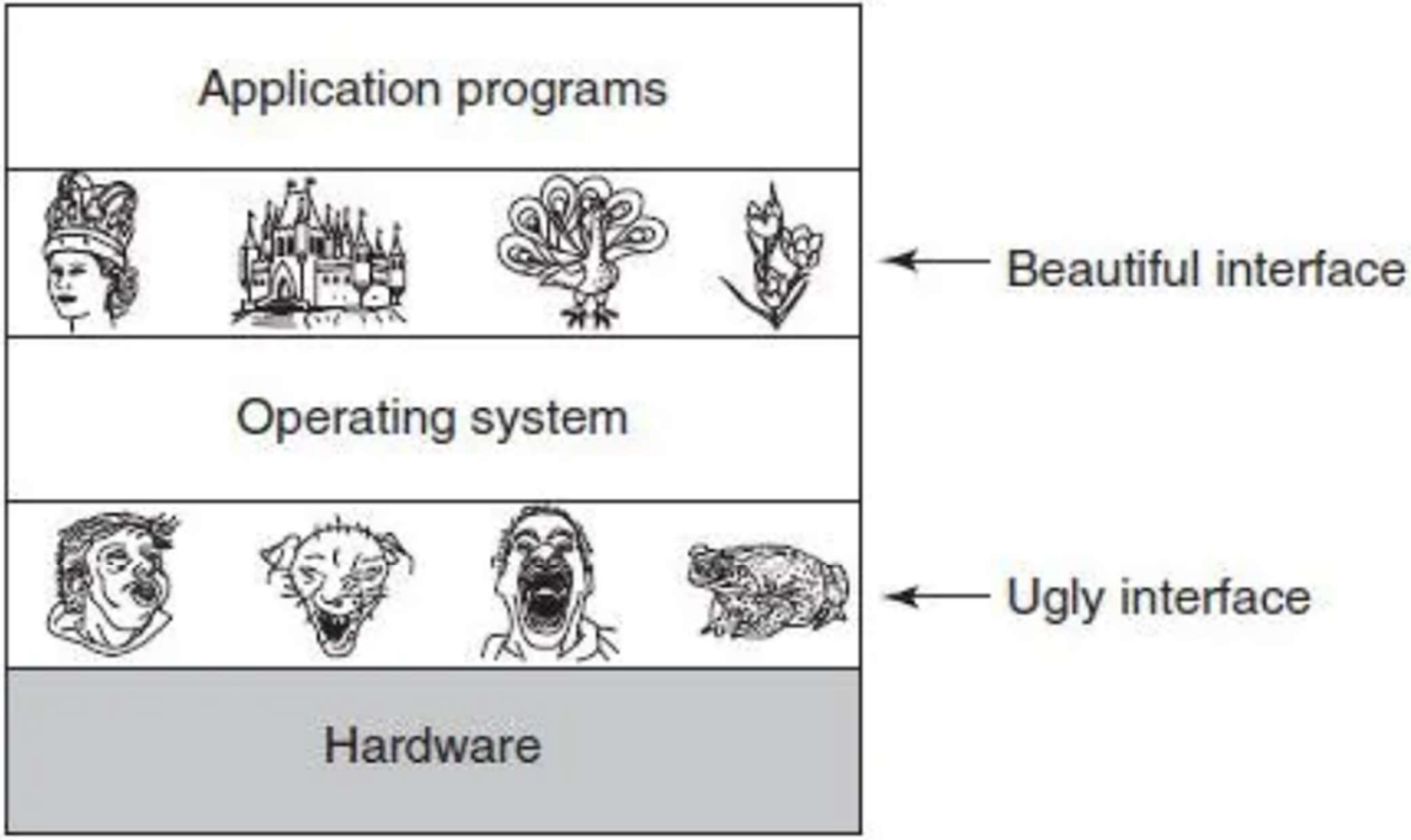
INHA UNIVERSITY IN TASHKENT

SOC 3010 OPERATING SYSTEMS

Processes and Signals, Completed

INSTRUCTOR: Jalolliddin Yusupov

What is an Operating System?



Disclaimer: this slide and the following three are freely inspired from:
http://lass.cs.umass.edu/~shenoy/courses/spring10/lectures/Lec01.pdf

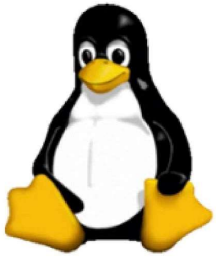
Why do we need to study Operating Systems?

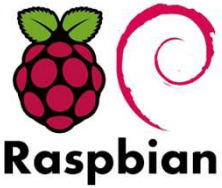
- **Abstraction**: To understand how the OS can give users the illusion of infinite memory, CPUs, resources, network, etc.
- **Better application and system design**: to understand how to design your application, based on how the OS works (e.g., use an OS primitive instead of another, use OS functions or re-implement from scratch in your app, etc.)
  - Applications delegate many functions to the OS by calling kernel-level primitives ("system calls", http://man7.org/linux/man-pages/man2/syscalls.2.html)
  - To create apps that **use your computers more effectively**.
    - *Example: sending to the network a byte at a time is much less efficient that sending the entire buffer at once.*
- **Learn widely-used mechanisms**: many concepts seen in the OS course (concurrency, synchronization, deadlocks, etc.) will be heavily used in **applications** and are a "must" for programmers, and which often require explicit support from the OS

Why many operating systems?

- Different objectives
  - Be user-friendly
  - Be compact (for embedded devices, such as fridge, etc.)
  - Be robust (for critical devices, such as cars, etc.)
  - Be real-time
  - Be open, or Create lock-in for users (e.g., Microsoft, Apple)
  - Etc.















SOC 2040 - System Programming



SOC 3010 - Operating Systems

- learned how to be a power user of the system
  - in the driver's seat
  - how to write C code
  - make system calls
  - tell the OS what to do
  - treated the OS like a black box
    - a helpful assistant that granted your requests for memory, file access, or network connections.

We are opening up that black box

- no longer just the driver
- you are the mechanic and the engineer
- aren't just going to ask the OS to do things anymore
- figure out exactly *how* it does them
- how it prevents programs from crashing into each other
- how it manages limited hardware resources behind the scenes

Time for a Quick Recap!

Before we move forward, let's review what we remember from the last lectures.

Grab your phone or open a new browser tab.

Go to **livepoll.io** or simply scan the QR code on the screen.

Answer a few quick questions to test your knowledge of the previous course material.

**Note: This activity is highly encouraged, but not compulsory.*

SOC 3010 - OPERATING SYSTEMS **TOPIC FLOW**

W1

Processes & signals HISTORY

exec · sigaction · review

System Programming | OSTEP 4–6

⏮ Review from SOC 2040 — fork, exec, signal handlers done properly.

W2

The shell & scheduling

bash · job control · MLFQ, CFS

TLPI 34 | OSTEP 7–8

W3–4

IPC combined

Pipes · FIFOs · MQ · shm · semaphores

TLPI 44–54 | dup2 plumbing | POSIX-first

Pipes → message queues → shared memory — the padlock.

W5

Sockets & servers

fd passing · fork · select

TLPI 56–61

W6–7

Semaphore patterns combined

multiplex · signalling · buffer · R/W · barber · philosophers

OSTEP 31 | Downey | Coffman

Classical synchronisation problems with semaphores.

W8

Midterm EXAM

26–30 Oct

25%

Covers processes, IPC, semaphores.

W9

Threads & locks

races · TAS/CAS · futex

OSTEP 26–28

W10

Condvars & rwlocks

the two idioms · pools

OSTEP 30–31

↔ Semaphore trio → condition-variable idiom (drawn).

W11

Deadlock & events

Coffman · lock order · epoll

OSTEP 32–33

select/poll → epoll event loop.

W12–13

Memory I & II combined

paging · TLB · page faults · replacement · malloc · OOM

OSTEP 13–22

Virtual memory illusion → pressure → the OOM Killer.

W14

Files & FS

inodes · links · access paths

OSTEP 39–40

fd → open-file → inode triple completed.

W15–16

Final Exam & Project Fair FINAL

21–25 Dec · review · container from scratch

25% + 20% | OSTEP 42 + supp.

⭐ Grand review: build a container from scratch.
W15 originally "fsync & containers" → replaced with Final Exam.

Program vs. process

A program is a file — inert, cold, waiting.
A process is the file *running*: registers, an address space, open files, a PID, and a "criminal" record (signals pending).

two bash windows → one program, two processes.

> *"The same recipe, baked twice — different kitchens, different chefs."*

CPU illusion · memory illusion

The Two Lies the Kernel Tells

Lie #1 — Logical control flow (scheduling, W2)

Every process believes it has the CPU all to itself. The kernel juggles, swaps, preempts — and the process never notices.

Lie #2 — Private address space (paging, W12)

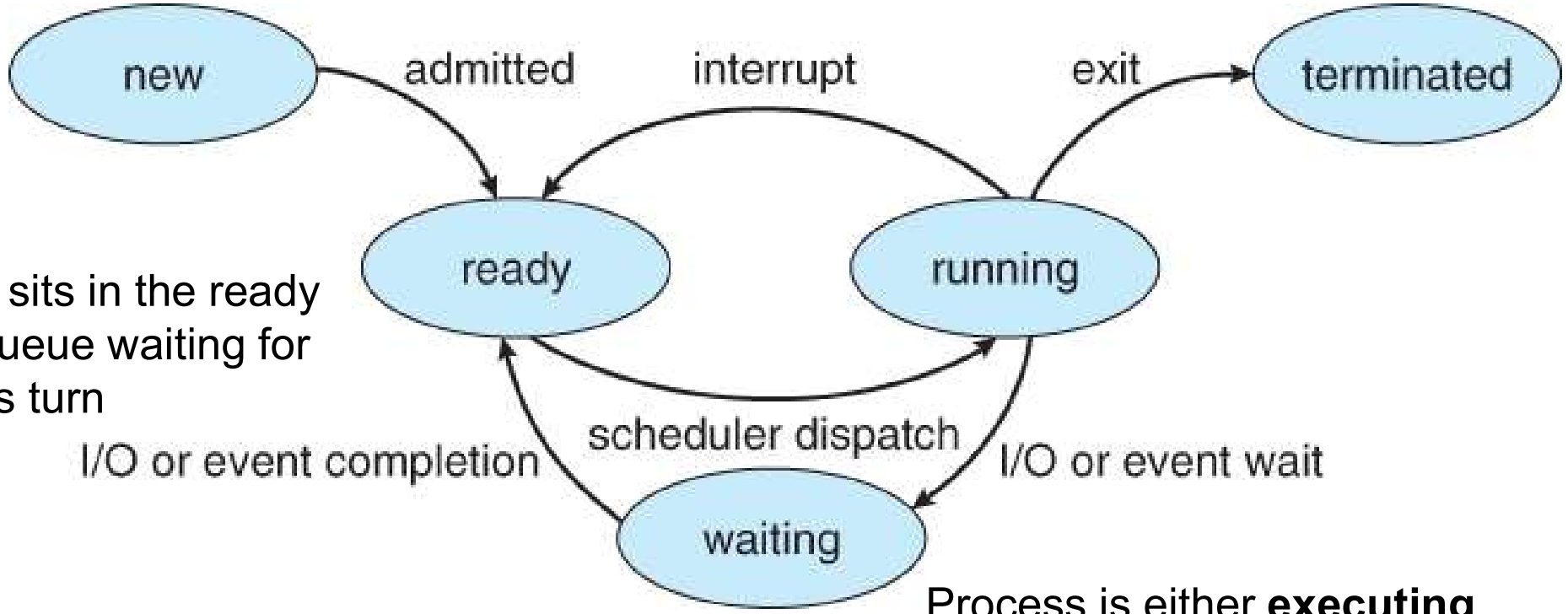
Every process believes it owns all of memory. In reality, it's a carefully stage-managed set of pages.

> *"Every process is told it's an only child. The kernel maintains this lie until food runs out (W13)."*

Process States

The process has just been created - **fork()**

(1) receiving a signal
(2) returning from the main routine
(3) calling the exit function



It sits in the ready queue waiting for its turn

Process is either **executing**, or **waiting** to be executed and will eventually be scheduled (i.e., chosen to execute) by the kernel

fork() - one call, two returns

`fork()` returns:

- **0** → in the child
- **child's PID** → in the parent
- **−1** → no child created; `errno` says why

> *"Always check −1 — the one branch students never write is the one production hits."*

What fork() Actually Copies

- **fd table** — same open files, shared offsets (W3's pipes depend on this)
- **signal dispositions** — handlers inherited
- **memory image** — *copy-on-write*: page tables copied, pages shared read-only until someone writes (the bill arrives in W12)
- **pending-signal set** — cleared in the child

Nondeterminism is the Default

parent and child are two runnable processes

Who prints first is the scheduler's mood (W2). Run it twenty times; collect the orders.

waitpid(pid, &status, options)

waitpid() function suspends the execution of a calling process until a specific child process changes its state (such as terminating or stopping) **or a signal received**

- `pid > 0` — that specific child
- `pid = -1` — any child
- `pid = 0` — any child in my process group
- `pid < -1` — any child in group |pid| (W2's job control)

`wait(&st)` ≡ `waitpid(-1, &st, 0)`.

Ctrl-C → SIGINT

Ctrl- → SIGQUIT (with a core dump)

Ctrl-Z → SIGTSTP

A Signal Is a Doorbell

asynchronous, one integer, no payload

`kill -l` — the kernel and other processes ring it.

> *"Signals are doorbells, not conversations — you cannot send a filename through SIGUSR1."*

(TRY THIS OUT) Installing Signal Handlers

```c
// multi_signal.c
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
void handler(int sig) {
    switch(sig) {
        case SIGINT:
            printf("\nCaught SIGINT (Ctrl+C)\n");
            break;
        case SIGTERM:
            printf("\nCaught SIGTERM (termination request)\n");
            break;
        case SIGUSR1:
            printf("\nCaught SIGUSR1 (user-defined 1)\n");
            break;
    }
}
int main() {
    printf("Process %d waiting for signals...\n", getpid());
    printf("Try: Ctrl+C (SIGINT) or kill -TERM %d (SIGTERM)\n\n", getpid());
    signal(SIGINT, handler);
    signal(SIGTERM, handler);
    signal(SIGUSR1, handler);
    while(1) {
        pause();  // Wait for signals
    }
    return 0;
}
```



Questions?